
Concurrencia y Paralelismo

Bloque I: Concurrencia

Julio 2023

1. Poetas [2.5 puntos]

Un grupo de N poetas se sienta en una mesa formando una fila, numerados de 0 a $N-1$. Disponen de un único bolígrafo, por lo que solo una persona puede estar escribiendo a la vez. El número de quien tiene el bolígrafo está indicado por el campo `pen_holder`, y el número de a quien le toca escribir por `current_poet`.

Cuando la persona indicada por `current_poet` recibe el bolígrafo escribe durante un tiempo, representado por una llamada a la función `write()`, que debe hacerse sin ningún `mútex` bloqueado. Cuando finaliza, escoge a la siguiente persona que recibirá el bolígrafo generando un número aleatorio entre 0 y $N-1$, pero garantizando que no es ella misma.

Como están sentados en la mesa formando una fila, para hacer llegar el bolígrafo hasta esa persona tiene que irse pasando por todas las personas que estén entre ellas. Por ejemplo, para pasar el bolígrafo de la persona 5 a la 8, la 5 se lo pasaría a la 6, la 6 a la 7, y la 7 a la 8. Hay que tener en cuenta que el bolígrafo puede tener que ir hacia el final o hacia el principio de la mesa.

Cuando una de las personas no tenga que escribir ni pasar el bolígrafo deberá estar esperando. No deben usarse esperas activas, ni despertar a más de una persona a la vez.

Defina la función `poet` de tal manera que se comporte de la forma descrita.

```
#define N ...

void write(); // Call this function to simulate the writing
int rand();  // generate a random number between 0 and RAND_MAX

typedef struct {
    int current_poet; // starts with a valid value from 0 to N-1
    int pen_holder    // which poet has the pen right now.
    ...               // starts with the same value as current_poet
} poet_group;

typedef struct {
    int id; // poet number
    poet_group *group; // shared structure
    ...
} poet_info;

void poet(poet_info *inf) {

}
```

```
int mtx_lock(mtx_t *mtx);
int mtx_trylock(mtx_t *mtx);
int mtx_unlock(mtx_t *mtx);
int cnd_wait(cnd_t *cond, mtx_t *mtx);
int cnd_signal(cnd_t *cond);
int cnd_broadcast(cnd_t *cond);
```

Solución:

```
#define N ...

void write(); // Call this function to simulate the writing
int rand();  // generate a random number between 0 and RAND_MAX

typedef struct {
    int current_poet; // starts with a valid value from 0 to N-1
    int pen_holder    // which poet has the pen right now.
                      // starts with the same value as current_poet
    mtx_t lock;
    cnd_t waiting[N];
} poet_group;

typedef struct {
    int id; // poet number
    poet_group *group; // shared structure
} poet_info;

void poet(poet_info *inf) {
    while(true) {
        mtx_lock(&inf->group->lock);
        while(inf->group->pen_holder != inf->id)
            cnd_wait(&inf->group->waiting[inf->id], &inf->group->lock);
        if(inf->group->current_poet == inf->group->id) {
            mtx_unlock(inf->group->lock);
            write();
            mtx_lock(inf->group->lock);
            while(inf->group->current_poet == inf->id)
                inf->group->current_poet = rand() % N;
        }
        if(inf->group->current_poet > inf->id)
            inf->group->pen_holder++;
        else
            inf->group->pen_holder--;
        mtx_signal(&inf->group->waiting[inf->group->pen_holder]);
        mtx_unlock(&inf->group->lock);
    }
}
```

2. Pool de recursos [2.5 puntos]

Un grupo de threads necesita usar una unidad de un recurso del cual hay unidades limitadas, pero que no se consume al usarlo. Para manejar esos recursos se utiliza una pila almacenada en un array con un número que marca la cima.

Cuando un thread necesita hacer una operación, tiene que comprobar si hay alguno disponible. Si es así, lo retira de la estructura, lo usa, y lo vuelve a insertar para que los siguiente puedan utilizarlo. Si no hay ninguno libre debería esperar a que lo haya sin usar esperas activas.

Implemente la función `worker` con el comportamiento descrito.

```
typedef struct {
    resource *pool[...];
    int top; // top of the resource stack
    ...
} resource_pool;

void worker(resource_pool *p) {
    resource *r;

    ...
    use(r);
    ...
}
```

Solución

```
typedef struct {
    resource *pool[...];
    int top; // top of the resource stack
    mtx_t lock;
    cnd_t empty;
} resource_pool;

void worker(resource_pool *p) {
    resource *r;

    mtx_lock(&p->lock);
    while (p->top == -1)
        cnd_wait(&p->empty, &p->lock);
    r = p->pool[p->top];
    p->top--;
    mtx_unlock(&p->lock);

    use(r);

    mtx_lock(&p->lock);
    p->top++;
    p->pool[p->top] = r;
    if(p->top == 0)
        mtx_broadcast(&p->empty);
    mtx_unlock(&p->lock);
}
```
